

CIVICALERT

Government Incident Reporting & Emergency SOS System

FULL SYSTEM ARCHITECTURE

Software Architecture • Database • API • Security • Mobile • Web • SOS • Notifications • Deployment • Testing

Prepared as a complete implementation blueprint for a government incident-reporting platform supporting anonymous reporting, geolocation, evidence uploads, authority routing, and emergency SOS.

Document Version: 1.0
Generated: 30 August 2026
Status: Architecture & Build Specification

Table of Contents

#	Section
1	Project Overview
2	System Scope
3	User Roles
4	Complete System Architecture
5	Citizen Incident Reporting Flow
6	SOS Architecture and Flow
7	Authority Routing Architecture
8	Location and Mapping Architecture
9	File Upload Architecture
10	Identity, Anonymous Reporting and IP Metadata
11	Notification Architecture
12	Government Administration Portal
13	Database Architecture
14	API Architecture
15	Security Architecture
16	Offline and Mobile Architecture
17	Deployment Architecture
18	Monitoring and Operations
19	Testing Strategy
20	Development Roadmap
21	MVP Definition
22	Production Readiness Checklist

1. Project Overview

CivicAlert is a government incident-reporting and emergency response platform. Citizens can submit public incidents using a web portal or mobile application. Reports can include descriptions, photos, documents, geographic coordinates and other supporting data.

The platform automatically determines the responsible government authority using the incident category and geographic administrative area. The report is then routed to the appropriate department and notification is delivered through email.

The mobile application additionally provides an SOS mechanism. When the supported device-level SOS trigger is activated, the application creates an emergency event and attempts to notify the configured emergency contact by email, SMS and phone call.

The system is designed around security, auditability, privacy, geographic data, notification reliability and government workflow management.

2. System Scope

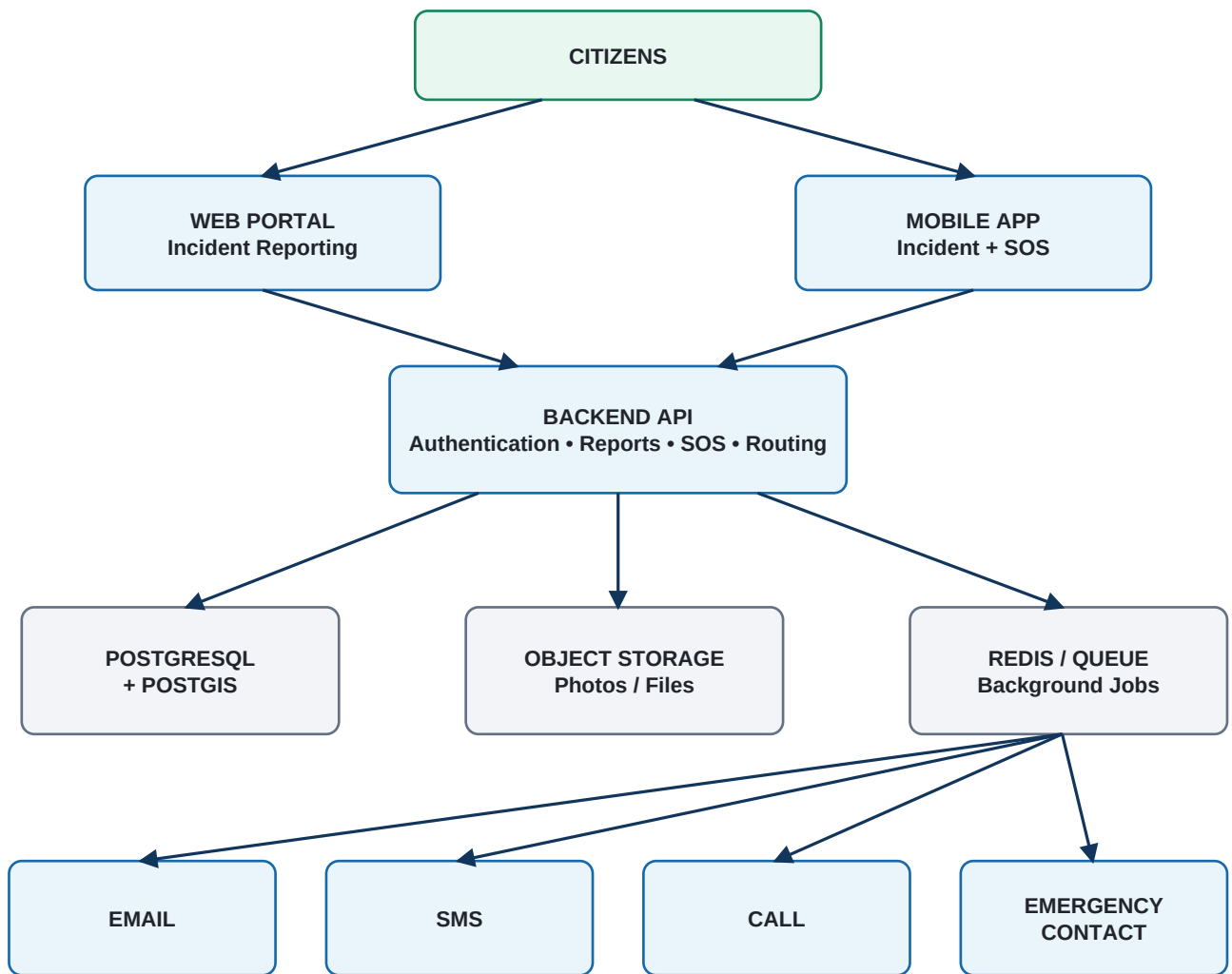
Area	Capabilities
Citizen Web	Incident reporting, anonymous/identified mode, location, attachments, tracking.
Citizen Mobile	Incident reporting, GPS, camera, emergency contacts, SOS.
Government Portal	Reports, assignments, departments, authority routing, SOS monitoring, analytics.
Backend	REST API, authentication, routing, notification orchestration, audit.
Database	PostgreSQL + PostGIS for relational and geographic data.
Storage	Private object storage for photos, videos and documents.
Notifications	Email, SMS and phone-call integrations.
Queue	Redis-backed asynchronous processing for notifications and heavy jobs.
Security	RBAC, MFA for administrators, encryption, audit logs, rate limiting.

3. User Roles

Role	Responsibilities
Citizen	Submit incidents, track reports, manage emergency contact, trigger SOS.
Department Officer	Review assigned reports, update status, add notes, take action.
Department Administrator	Manage department officers and routing information.
System Administrator	Manage users, departments, authorities, categories, configuration and audit.

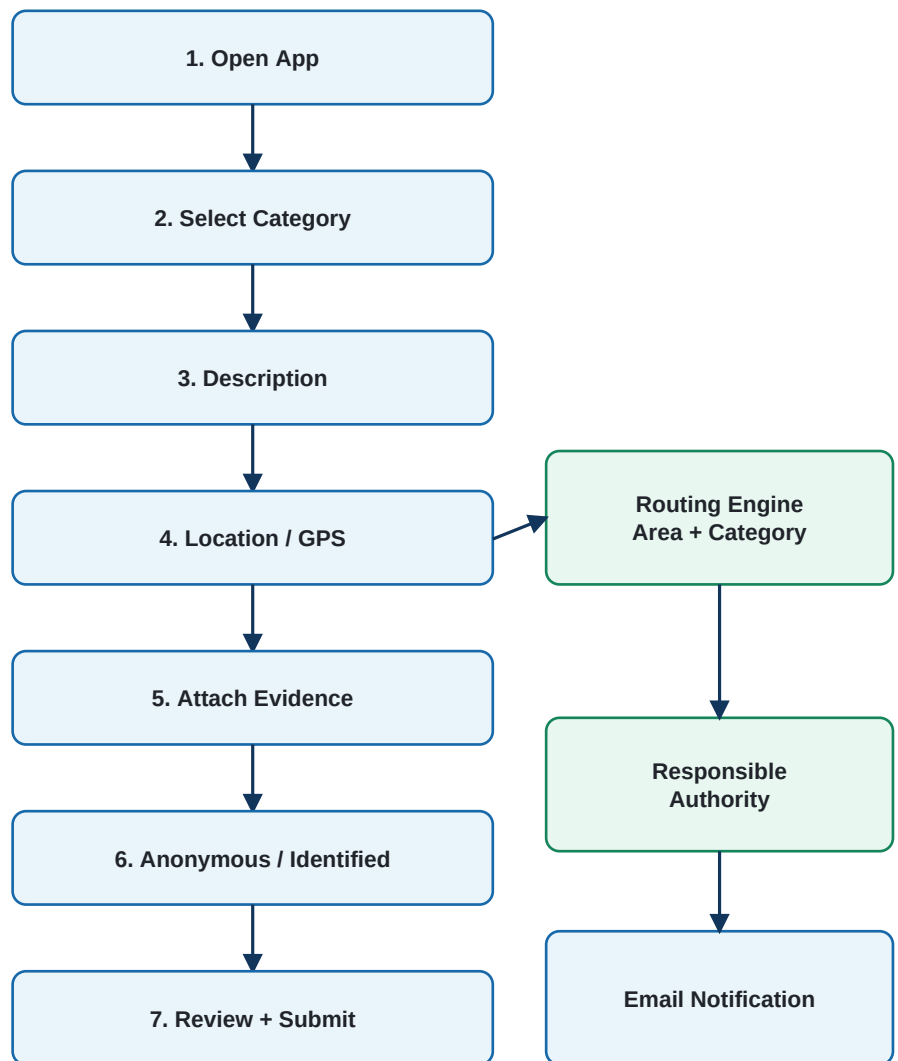
4. Complete System Architecture

The system is separated into presentation, application, data, notification and administration layers. The mobile application provides device-specific functionality while the web application provides browser-based reporting and tracking.



Architecture principle: the client applications must never contain database credentials, email credentials, SMS credentials, calling-provider secrets or authority-routing secrets. All privileged operations pass through the backend API.

5. Citizen Incident Reporting Flow



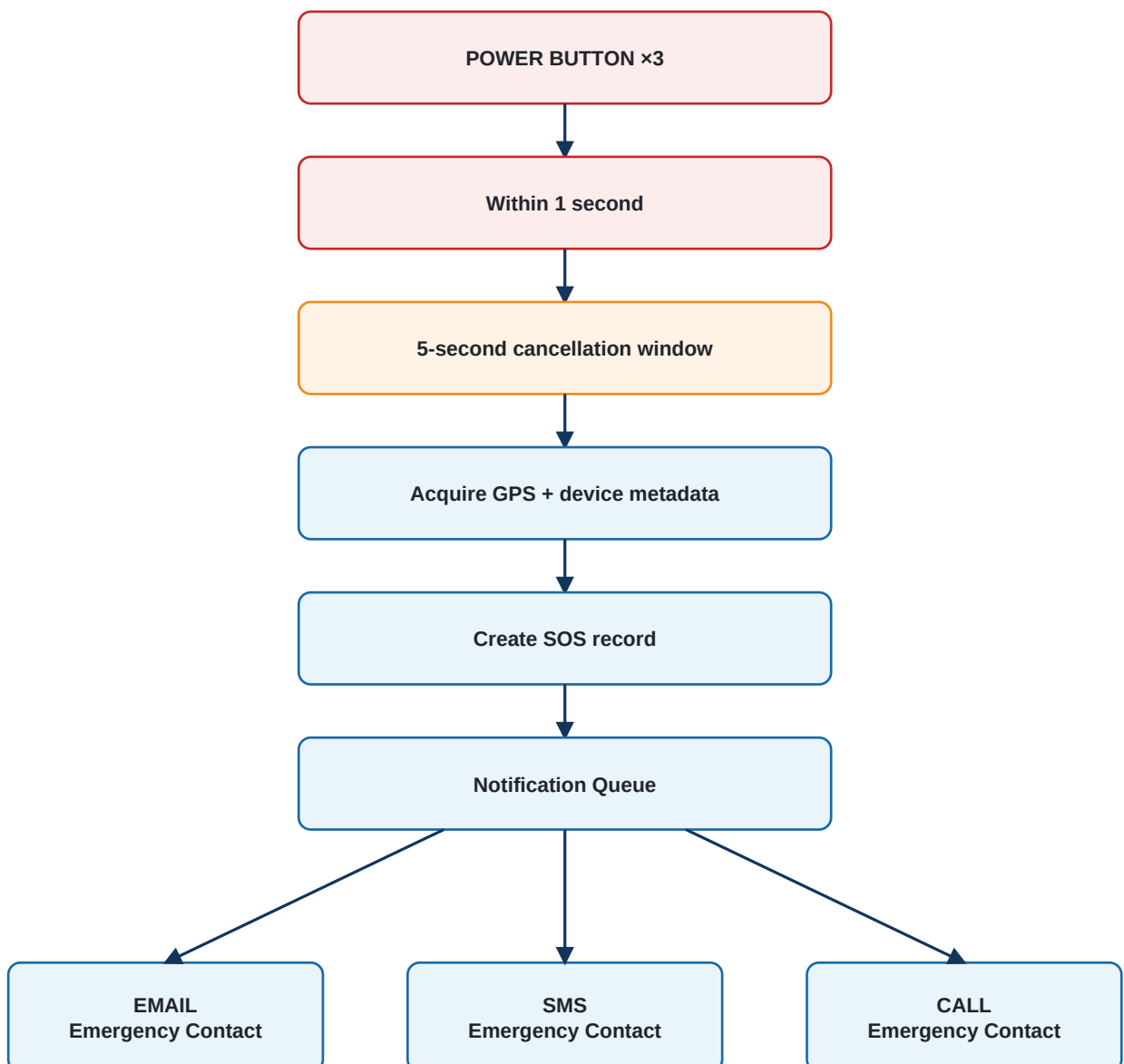
A report begins as a draft and becomes a permanent incident record only after successful server-side validation. The server generates the reference number rather than trusting a client-generated ID.

Step	Action
1	Citizen opens the application.
2	Citizen selects an incident category.
3	Citizen enters title and description.
4	Citizen captures GPS location or manually selects a map point.
5	Citizen optionally attaches photos, video or documents.
6	Citizen chooses anonymous or identified reporting.
7	System validates and stores the report.
8	Routing engine determines responsible authority.

Step	Action
9	Notification job is queued.
10	Authority email is sent and delivery result recorded.
11	Citizen receives report reference number.

6. SOS Architecture and Flow

SOS is an emergency workflow and should be isolated from ordinary incident processing. The mobile application initiates the SOS event, while the backend coordinates notification delivery.



Important implementation constraint: a standard web browser cannot reliably detect physical power-button presses. The requested three-press trigger therefore belongs in the mobile application and must be implemented/tested against the supported operating system and device APIs. The backend should expose a

testable SOS endpoint so the entire notification pipeline can be developed independently of the device trigger.

Stage	System Behavior
Trigger	Supported mobile-device SOS mechanism detects the configured trigger.
Debounce	Three presses must occur within the configured time window.
Cancellation	A short cancellation window reduces accidental activation.
Location	Application attempts to acquire current location.
Record	Backend creates a unique SOS reference.
Queue	Notification jobs are created independently.
Email	Emergency-contact email is attempted.
SMS	Emergency-contact SMS is attempted.
Call	Emergency-contact phone call is attempted.
Audit	Each notification result is recorded.

Notification channels must be independent. A failed email must not prevent the SMS or phone call from being attempted.

7. Authority Routing Architecture

Authority routing should be database-driven. Government administrators must be able to update responsible departments and contact addresses without requiring a software deployment.

Input	Processing	Output
GPS coordinates	Administrative boundary lookup	Province / district / municipality / ward
Incident category	Routing rule lookup	Responsible department
Area + department	Authority lookup	Email / phone / notification destination

```
GPS Coordinates
↓
Administrative Boundary
↓
Municipality / Ward
↓
Incident Category
↓
Routing Rule
↓
Responsible Authority
↓
Notification
```

8. Location and Mapping Architecture

Location data is a core system capability. The mobile application should request location permission only when needed and should clearly explain why the location is being collected.

Field	Purpose
latitude	Geographic latitude.
longitude	Geographic longitude.
accuracy	Reported GPS accuracy in meters when available.

Field	Purpose
address	Human-readable address.
administrative_area	Resolved government administrative area.
captured_at	Time at which location was captured.

PostGIS should be used for geographic queries, including determining which administrative boundary contains a submitted coordinate and supporting map-based incident analysis.

9. File Upload Architecture

```

Client
↓
Upload API
↓
Authentication + authorization
↓
Extension / MIME / content validation
↓
File-size validation
↓
Malware scanning
↓
Random server-side filename
↓
Private object storage
↓
Attachment database record

```

Requirement	Implementation
File types	Explicit allowlist of supported image/video/document types.
File size	Server-side maximum size.
Filename	Generate random storage keys; never trust user filenames.
Storage	Private object storage, not publicly accessible.
Access	Short-lived authorized download URLs or backend proxy.
Security	Validate actual content and scan where infrastructure permits.

10. Identity, Anonymous Reporting and IP Metadata

Citizens may report anonymously or provide their identity. Anonymous reporting means the reporter's name, email and phone number are not included in the authority-facing report.

Mode	Citizen Data	Authority View
Anonymous	No identity fields required	Anonymous
Identified	Name, phone, email where provided	Reporter identity according to permissions

The system may retain technical metadata such as IP address, user-agent, session/device identifiers and timestamps for security, abuse prevention and audit purposes where legally permitted.

IP addresses must not be treated as a reliable identity. Mobile carriers, NAT, VPNs, shared networks and changing addresses can result in multiple users sharing an IP or one user changing IPs.

```

Request
■■■ Report data
■■■ IP address
■■■ User-Agent

```



```

■■■ Session / device identifier
■■■ Timestamp
↓
Secure technical audit metadata

```

11. Notification Architecture

Email, SMS and calling should be implemented behind a common notification service. Asynchronous queues should be used for notifications so an external provider failure does not block the main report transaction.

Notification	Normal Report	SOS
Email	Responsible authority	Emergency contact
SMS	Optional	Emergency contact
Call	No	Emergency contact

[illegible]

12. Government Administration Portal

Module	Features
Dashboard	Total reports, pending, in progress, resolved, active SOS.
Reports	Search, filtering, assignment, status changes, notes.
Map	Incident markers, filters, geographic analysis.
SOS	Active SOS events, notification status, location.
Authorities	Departments, contact addresses, areas.
Routing	Category + geographic routing rules.
Users	Officer and administrator management.
Analytics	Category, area, response and resolution statistics.
Audit	Security and administrative activity logs.

Government users should only see information required for their role. For example, ordinary department officers should not automatically receive unrestricted access to sensitive technical metadata.

13. Database Architecture

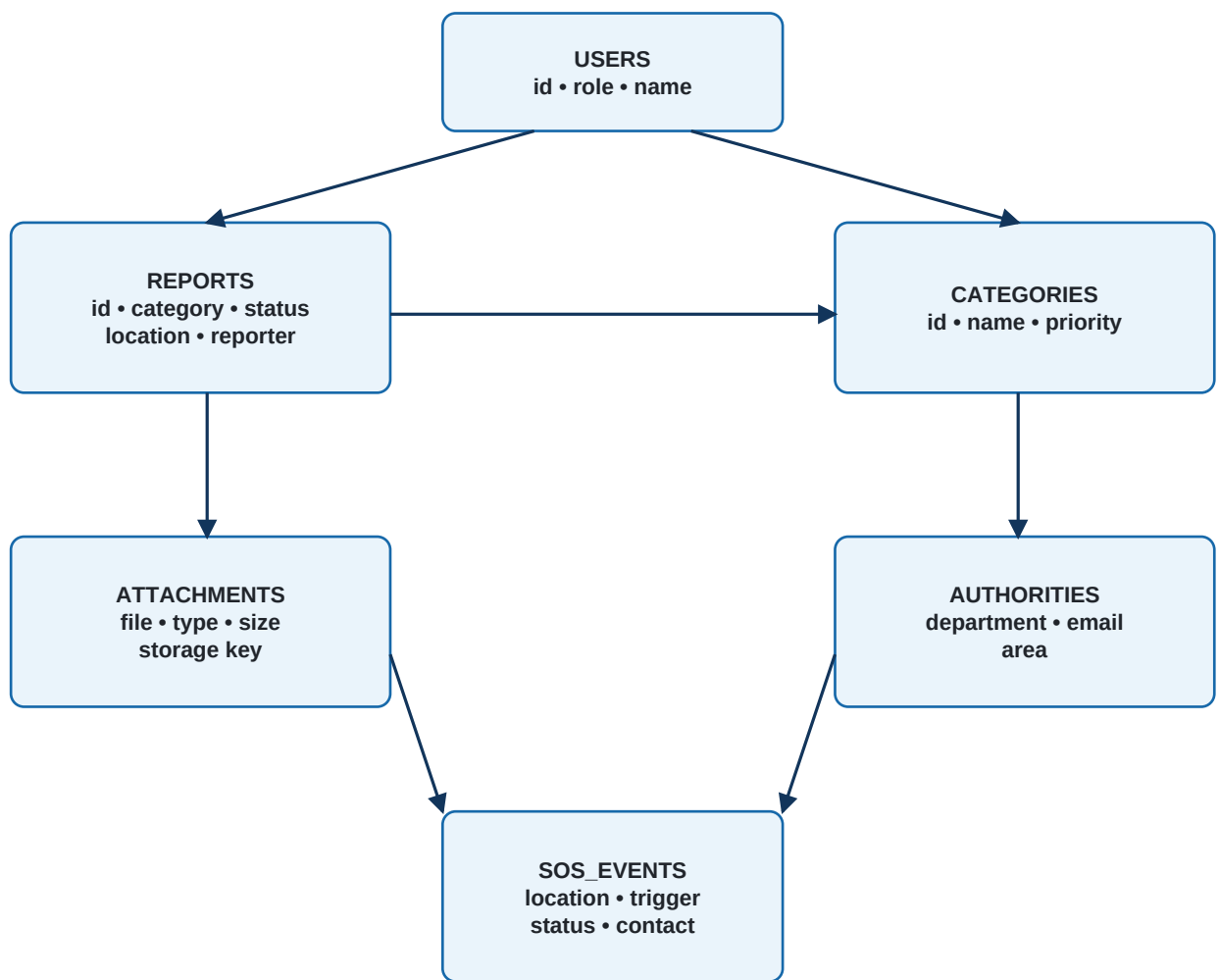


Table	Important Fields
users	id, role_id, name, email, phone, password_hash, status
roles	id, name
reports	id, reference_number, user_id, anonymous, category_id, title, description, location, priority, status, authority_id
incident_categories	id, name, description, icon, default_priority, active
authorities	id, name, department, email, phone, area, active
attachments	id, report_id, file_name, storage_key, mime_type, file_size
report_status_history	id, report_id, old_status, new_status, changed_by, note, created_at
emergency_contacts	id, user_id, name, phone, email, relationship, is_primary
sos_events	id, reference_number, user_id, device_id, ip_address, latitude, longitude, trigger_type, contact_id, status
sos_notifications	id, sos_id, type, recipient, status, provider_reference, sent_at
audit_logs	id, user_id, action, entity_type, entity_id, ip_address, created_at

14. API Architecture

Method	Endpoint	Purpose
POST	/api/auth/register	Create citizen account
POST	/api/auth/login	Authenticate
POST	/api/reports	Create incident report
GET	/api/reports/:id	Get report
GET	/api/reports	List permitted reports
PATCH	/api/reports/:id	Update report
POST	/api/reports/:id/attachments	Upload evidence
POST	/api/reports/:id/status	Change status
GET	/api/categories	List incident categories
GET	/api/authorities	List authorities
GET	/api/emergency-contact	Get emergency contact
POST	/api/emergency-contact	Create emergency contact
PATCH	/api/emergency-contact	Update emergency contact
POST	/api/sos	Create SOS event
GET	/api/sos/:id	Get SOS event
POST	/api/sos/:id/cancel	Cancel SOS where permitted
GET	/api/admin/dashboard	Admin dashboard
GET	/api/admin/reports	Administrative report list
GET	/api/admin/sos	Administrative SOS list
GET	/api/admin/audit-logs	Audit records

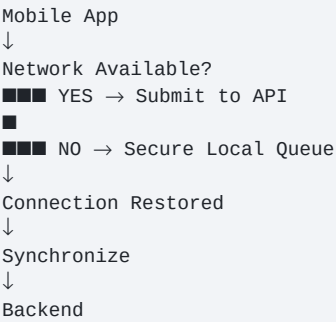
15. Security Architecture

- TLS/HTTPS for all client-to-server communication.
- Password hashing using a modern password-hashing algorithm.
- Short-lived access tokens and secure refresh-token handling.
- Multi-factor authentication for government administrators.
- Role-based access control on every protected backend operation.
- Server-side validation of all user input.
- Parameterized database queries / ORM protections.
- CSRF protection where cookie-based authentication is used.
- Rate limiting on authentication, reporting and SOS endpoints.
- Strict file-upload validation and private file storage.
- Secrets stored in environment/secret-management infrastructure, never source code.
- Audit logging for administrative and sensitive operations.
- Encryption at rest for sensitive data where supported.
- Database backups with access controls.

- Data-retention and deletion policies.
- Security monitoring and incident-response procedures.

Security priority: anonymous reporting, location data, IP metadata, emergency contacts and government administrative data should be treated as sensitive information and protected using least-privilege access.

16. Offline and Mobile Architecture



Ordinary incident reports may be stored locally while offline and uploaded when connectivity returns. Emergency behavior must clearly report which notification channels succeeded or failed.

Feature	Mobile Responsibility
GPS	Request permission, capture coordinates and accuracy.
Camera	Capture evidence directly.
Storage	Securely queue unsent data where necessary.
SOS	Initiate supported device-level emergency trigger.
Contacts	Store/manage configured emergency contact.
Notifications	Display report and SOS delivery status.

17. Deployment Architecture



Component	Purpose
WAF / Firewall	Protect public application endpoints.
Load Balancer	Distribute traffic across application instances.
Web Frontend	Serve citizen and administrative interfaces.
API	Business logic and secure data access.

Component	Purpose
PostgreSQL/PostGIS	Transactional and geographic data.
Redis	Queue, caching and short-lived coordination.
Object Storage	Private evidence/file storage.
Workers	Asynchronous email, SMS, call and processing jobs.
Monitoring	Logs, metrics, errors and service health.
Backup	Database and object-storage backup/recovery.

18. Monitoring and Operations

- API availability and response time.
- Database health and connection utilization.
- Queue length and failed jobs.
- Email delivery failures.
- SMS delivery failures.
- Call initiation failures.
- SOS processing time.
- Authentication failures.
- Suspicious activity and rate-limit events.
- Storage usage.
- Backup success/failure.

SOS processing should be observable end-to-end: SOS received, record created, email attempted, SMS attempted, call attempted, and final delivery status.

19. Testing Strategy

Test Level	Scope
Unit	Routing, validation, permissions, priority and notification logic.
API	Authentication, authorization, report lifecycle, SOS endpoints.
Integration	Database, storage, routing and notification providers.
E2E	Citizen report from creation through government resolution.
Mobile	GPS, camera, background behavior, offline operation and SOS.
Security	Authentication, authorization, file uploads, injection, abuse.
Performance	Concurrent reports, uploads and notification queues.
Recovery	Database restore, queue recovery and service failure.

SOS Test Matrix

Scenario	Expected Result
App foreground	SOS trigger processed.

Scenario	Expected Result
App background	Supported behavior tested and documented.
Screen locked	Supported behavior tested on target devices.
GPS available	Current location included.
GPS unavailable	SOS still recorded; location marked unavailable.
Internet unavailable	Fallback behavior clearly handled.
SMS failure	Email/call continue independently.
Email failure	SMS/call continue independently.
Call failure	Email/SMS results remain recorded.
Accidental trigger	Cancellation window prevents accidental alert.
Repeated trigger	Cooldown/rate limiting prevents notification abuse.

20. Development Roadmap

Phase	Deliverable
01	Requirements and stakeholder analysis
02	UI/UX design and prototype
03	Database schema and migrations
04	Backend foundation and authentication
05	Citizen web incident reporting
06	Maps, GPS and administrative boundaries
07	File upload and object storage
08	Authority routing engine
09	Email notification service
10	Government administration portal
11	Report tracking and status workflow
12	Mobile application
13	Emergency contact management
14	SMS integration
15	Calling integration
16	SOS backend and notification orchestration
17	Device-level SOS implementation/testing
18	Security hardening
19	Integration, performance and penetration testing
20	Deployment, pilot and production rollout

21. MVP Definition

The MVP should demonstrate a complete vertical slice rather than attempting every possible government feature.

- Citizen incident submission.
- Anonymous and identified reporting.
- GPS/map location.
- Photo/file attachment.
- Reference number.
- Automatic authority routing.
- Authority email notification.
- Report tracking.
- Government dashboard.
- Report assignment and status.
- Emergency contact configuration.
- SOS backend.
- SOS email notification.
- SOS SMS notification.
- SOS phone-call initiation.
- Technical audit metadata including IP where legally permitted.

22. Production Readiness Checklist

Area	Requirement	Complete
Architecture	All services documented	■
Database	Migrations and backups tested	■
Security	RBAC implemented and tested	■
Security	Administrator MFA enabled	■
Security	File upload security tested	■
Privacy	Privacy policy approved	■
Privacy	Retention policy approved	■
Reports	Full report lifecycle tested	■
Maps	GPS and manual location tested	■
Routing	Administrative routing verified	■
Email	Delivery and failure handling tested	■
SMS	Delivery and failure handling tested	■
Calling	Call initiation tested	■
SOS	Device-level trigger tested	■
SOS	Cancellation tested	■
SOS	Notification independence tested	■
Performance	Load testing completed	■

Area	Requirement	Complete
Recovery	Backup restore tested	■
Monitoring	Alerts configured	■
Deployment	Staging environment validated	■
Operations	Administrator manual completed	■

Final Architecture Summary

CivicAlert should be implemented as a secure, modular government platform consisting of a citizen web portal, citizen mobile application, government administration portal and centralized backend services.

Normal incident reports follow the path:

Citizen → Report → Location → Authority Routing → Email → Government Officer → Resolution

Emergency events follow the path:

Mobile SOS → Location → SOS Record → Notification Queue → Email + SMS + Call → Emergency Contact

The architecture deliberately separates device-level functionality from browser functionality, keeps sensitive credentials on the server, uses asynchronous notification processing, stores geographic information using PostGIS, and provides auditability and role-based access for the government environment.

Recommended implementation principle: build and validate the normal incident-reporting pipeline first, then implement the SOS notification backend, and finally integrate/test the device-level three-press trigger on the supported mobile platform.